

# DBMS Lab

## Structured Query Language (SQL)

L T P C

BCSE302L - 3 0 0 3

BCSE302P - 0 0 2 1

**Dr. Shashank Mouli Satapathy**  
Professor, SCOPE



**VIT<sup>®</sup>**  
Vellore Institute of Technology  
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering  
Vellore Institute of Technology  
Vellore, Tamil Nadu - 632014, India

July 13, 2026



# Introduction

## Database

It is a repository for organizing and accessing information.



# Introduction

## Database

It is a repository for organizing and accessing information.

## DBMS

- Software designed to store, manipulate, and retrieve data in a database is called a **Database Management System (DBMS)**.
- Here, we focus exclusively on a specific (and by far the most common) type of DBMS, the **Relational DBMS (RDBMS)**.
- There are **many** relational DBMSs.
- Fortunately, relational DBMSs speak a common language called **SQL**.



# Introduction

## SQL

- SQL is correctly pronounced as *S – Q – L*; however, many people also pronounce it *see – kwel*.
- Using SQL, we can define, manipulate, and query our data.
- SQL is divided into three major parts.
  - **Data manipulation language (DML)** → used to store and retrieve data from the database.
  - **Data description language (DDL)** → used to define the structure of the data.
  - **Data control language (DCL)** → used to restrict access to data by certain users.



# SQL Access Procedure in Lab

## Windows System

- 1 Press  +  to open Run Window.
- 2 Type **sqlplus** and press 
- 3 In the command prompt, Type:
  - username: REGD.NO@SCOPEORA [ALL CAPS]
  - password: REGD.NO [ALL CAPS]
- 4 **sql>** will appear to type and execute queries.

## Linux System

- 1 Open Terminal
- 2 Type **sqlplus** and press 
- 3 In the command prompt, Type:
  - username: REGD.NO@SCOPEORA [ALL CAPS]
  - password: REGD.NO [ALL CAPS]
- 4 **sql>** will appear to type and execute queries.

# Tables

- In the relational model, a database contains a set of tables.
- A **table** is made up of **rows** and **columns**.
- Each **table** has a **name**, which is **unique** within the database.
- Each **column** has a **name** and a **data type**.
- The **name of a column** need only be **unique within a table** so other tables in the same database can have columns of the same name.
- Each **row** constitutes one **record** in the table.
- A **table** may contain **zero or more rows**.
- A **row** is subdivided into **fields**, one per column.
- Tables may be used to model real-world objects and relationships.
- A table may also be called a **relation**. Technically, a table and a relation are not exactly the same thing. **Unlike a table, a relation cannot contain duplicate rows.**



# Tables

- A **row** is sometimes called a **tuple**.
- Finally, a **column** may also be called an **attribute** of the relation.
- A **table** may be represented by its name followed by a comma-delimited list of columns:  
**vendors(vendorid, companyname, repfname, replname, referredby)**

| <b>vendors</b>  |                        |                 |                 |                   |
|-----------------|------------------------|-----------------|-----------------|-------------------|
| <b>vendorid</b> | <b>companyname</b>     | <b>repfname</b> | <b>replname</b> | <b>referredby</b> |
| VGRUS           | Veggies_R_Us           | Candy           | Corn            | NULL              |
| DNDRY           | Don's Dairy            | Marla           | Milker          | VGRUS             |
| FLVCR           | Flavorful Creams       | Sherman         | Sherbert        | VGRUS             |
| FRTFR           | "Fruit Eating" Friends | Gilbert         | Grape           | FLVCR             |
| EDDRS           | Ed's Dressings         | Sam             | Sauce           | FRTFR             |
| SPWTR           | Spring Water Supply    | Gus             | Hing            | EDDRS             |



# Tables

- Let's consider an example of **Restaurant Database**, we record information about the suppliers of our ingredients in a table named **vendors**.
- Each row in the **vendors** table records information about a particular vendor. For example, the row with a **vendorid** of **VGRUS** is a record representing the **vendor named Veggies\_R\_Us**.
- The **attributes** are represented by the five columns of the **vendors** table: a unique vendor identifier (**vendorid**), the name of the vendor (**companyname**), the first and last name of the vendor representative (**repfname** and **replname**), and an identifier for the vendor (**referredby**) who recommended this vendor.



# Data Types

- Every column has a declared data type, which specifies what kind of information may be contained in the column.
- SQL divides the data types into five categories: character string, numeric, temporal, binary, and boolean.

## Character String

- Attributes such as names and addresses are typically represented by strings of characters.
- There are many database types for character strings.
  - ❖ **CHARACTER (L)** → specifies a fixed-length character string containing exactly L characters.
  - ❖ If the length is not specified, SQL uses a length of 1.
  - \* If the string contains fewer than L characters, the remaining characters contain padding characters (usually spaces).
  - ❖ CHARACTER may be abbreviated as **CHAR**.



# Data Types

- ❖ **CHARACTER VARYING(L)** → specifies a variable-length character string that may hold up to L characters.
- ❖ Only the specified characters are stored so there is no padding.
- ❖ CHARACTER VARYING may be abbreviated as **VARCHAR**.
- ❖ **String literals** are enclosed in single quotes (e.g., 'Bob'). A single quote within a literal is indicated by two single quotes (e.g., 'Bob"s Car").
- ❖ The inclusion of trailing spaces is the primary difference between CHAR and VARCHAR. For example, **vendorid** is defined as **CHAR(5)** and **repfname** is defined as **VARCHAR(20)**.
- ❖ Assume we add the following row to **vendors** table:

|     |              |     |        |       |
|-----|--------------|-----|--------|-------|
| BOB | Bob's Bakery | Bob | Bobson | VGRUS |
|-----|--------------|-----|--------|-------|

- ❖ Because **vendorid** is **CHAR(5)**, two extra spaces are stored. However, **repfname** is **VARCHAR(20)**, so no extra spaces are added. This also makes a difference when we retrieve data from the database.



# Data Types

- ❖ The vendorid 'BOB\_\_' will match 'BOB' or 'BOB\_\_', but the repfname 'Bob' will not match 'Bob\_\_'.

## Numeric

- In specifying a numeric type, we need to consider three questions:
  - ➊ Are our data limited to whole numbers?
  - ➋ What range of values do we wish to support?
  - ➌ How much control do we need over precision?
- INTEGER, SMALLINT, and BIGINT → store signed whole numbers.
- The range of possible values is DBMS dependent; however, the range of values for SMALLINT is less than or equal to the range of value for INTEGER, which is less than or equal to the range of values for BIGINT.
- INTEGER may be abbreviated as INT.



# Data Types

- **NUMERIC[(P [, S])]** and **DECIMAL[(P [, S])]** → **NUMERIC(P, S)** specifies a signed, fixedpoint number where P (precision) specifies the total number (to the left and right of the decimal) of digits in the number and S (scale) specifies the number of digits to the right of the decimal place.
- For example, **NUMERIC(5, 2)** specifies a type ranging from -999.99 to 999.99.
- **DECIMAL(P, S)** is the same as **NUMERIC(P, S)** except that the actual precision may exceed the specification.
- For both **NUMERIC** and **DECIMAL**, default values will be used if P or S are not specified.
- **DECIMAL** may be abbreviated as **DEC**.
- **REAL** and **DOUBLE PRECISION** → **REAL** specifies a signed, single-precision, floating-point number.



# Data Types

- ▶ DOUBLE PRECISION is the same as REAL, except it supports a greater range of values.
- ▶ **FLOAT[(P)]** → FLOAT(P) specifies a signed, floating-point number with a precision of at least P. Here, P specifies the number of binary digits.

## Temporal

- ▶ Representing and utilizing date and time information can be complicated with considerations of time zone, daylight savings time, etc.
- ▶ SQL divides its temporal types into two categories: **datetime** and **interval**.
- ▶ **Datetime** → It stores date, time information, or both.



# Data Types

| Type             | Stores                                        | Literal                         |
|------------------|-----------------------------------------------|---------------------------------|
| <i>DATE</i>      | year, month, day                              | DATE 'YYYY-MM-DD'               |
| <i>TIME</i>      | hour, minute, and second                      | TIME 'HH:MM:SS'                 |
| <i>TIMESTAMP</i> | year, month, day, hour,<br>minute, and second | TIMESTAMP 'YYYY-MM-DD HH:MM:SS' |

- **Interval** → This might be useful for representing concepts such as a warranty period (e.g., 90 days).
- Interval data types come in two flavors:
  - **Year–Month–Interval** expressed in years and/or months. A Year–Month interval data type can be INTERVAL YEAR, INTERVAL MONTH, or INTERVAL YEAR TO MONTH.
  - **Day–Time–Interval** expressed in days, hours, minutes, and/or seconds.



# Data Types

- A Day–Time interval data type is of the following form:  
INTERVAL <start interval value> [TO <stop interval value>]

| Type              | Example Literal                       | Description                                    |
|-------------------|---------------------------------------|------------------------------------------------|
| <i>Year–Month</i> | INTERVAL '5' YEAR                     | 5 years                                        |
|                   | INTERVAL '2' MONTH                    | 2 months                                       |
|                   | INTERVAL '3-1' YEAR TO MONTH          | 3 years and 1 month                            |
| <i>Day–Time</i>   | INTERVAL '5 10:30:22.5' DAY TO SECOND | 5 days, 10 hours, 30 minutes, and 22.5 seconds |
|                   | INTERVAL '-5' DAY                     | 5 days ago                                     |
|                   | INTERVAL '2 18:00' DAY TO MINUTE      | 2 days and 18 minutes                          |

## Binary

- SQL binary data types are designed to store sequences of binary digits.
- Binary types differ from character string and numeric types in that they are more limited in the allowable comparisons, searches, and other functions.



# Data Types

- Common uses for binary types include storage of multimedia, such as photographs, sounds, and movies, and storage of scanned images, as in a document imaging and retrieval system.
- **BIT[(L)]** → specifies a fixed-length binary string containing exactly L bits. If the length, L, is not specified, SQL uses a length of 1.
- **BIT VARYING(L)** → specifies a variable-length binary string that may hold up to L bits.
- **BINARY LARGE OBJECT[(L)]** → specifies a large, variable-length binary string that may hold up to L bytes. If the length, L, is not specified, the system default length is used. BINARY LARGE OBJECT may be abbreviated as **BLOB**.

## Boolean

- The BOOLEAN data type has three possible values: TRUE, FALSE, and UNKNOWN.



# NULL

- Relational databases provide a special value, called **NULL**, indicating that a field's value is unknown.
- In the **vendors table**, **referredby** is **NULL** for all vendors where we do not have a recommending vendor.
- Unless explicitly forbidden, NULL is a valid value for a column of any data type.



# Primary Key

- The **primary key** of a table is a column or set of columns whose values uniquely identify a row. Using the values of the primary key, we can refer to a specific row in a table.
- In the **vendors** table, each vendor has a unique identifier, **vendorid**, which we use as the primary key.
- If we declare a primary key on a table, the DBMS will enforce the following rules:
  - The value of a primary key for a particular row must be unique.
  - None of the primary key column values can be NULL.



# Foreign Key

- A database typically contains several tables.
- Each table in a database usually has one or more relationships to other tables in the database.
- In addition to the **vendors** table, your restaurant application has a separate table of **ingredients**:

**ingredients**

| ingredientid  | name            | unit  | unitprice | foodgroup | inventory | vendorid  |
|------------------------------------------------------------------------------------------------|-----------------|-------|-----------|-----------|-----------|----------------------------------------------------------------------------------------------|
| CHESE                                                                                          | Cheese          | scoop | 0.03      | Milk      | 150       | DNDRY                                                                                        |
| CHIKN                                                                                          | Chicken         | strip | 0.45      | Meat      | 120       | DNDRY                                                                                        |
| CRUTN                                                                                          | Crouton         | piece | 0.01      | Bread     | 400       | EDDRS                                                                                        |
| GRAPE                                                                                          | Grape           | piece | 0.01      | Fruit     | 300       | FRTFR                                                                                        |
| LETUS                                                                                          | Lettuce         | bowl  | 0.01      | Vegetable | 200       | VGRUS                                                                                        |
| PICKL                                                                                          | Pickle          | slice | 0.04      | Vegetable | 800       | VGRUS                                                                                        |
| SCTDR                                                                                          | Secret Dressing | ounce | 0.03      | NULL      | 120       | NULL                                                                                         |
| TOMTO                                                                                          | Tomato          | slice | 0.03      | Fruit     | 15        | VGRUS                                                                                        |
| WATER                                                                                          | Water           | glass | 0.06      | NULL      | NULL      | SPWTR                                                                                        |
| SODA                                                                                           | Soda            | glass | 0.69      | NULL      | 5000      | SPWTR                                                                                        |
| WTRML                                                                                          | Watermelon      | piece | 0.02      | Fruit     | NULL      | FRTFR                                                                                        |
| ORNG                                                                                           | Orange          | slice | 0.05      | Fruit     | 10        | FRTFR                                                                                        |



# Foreign Key

- A **foreign key** is a column or set of columns in a table that refers to a column or set of columns in some table.
- In the Restaurant Database, **ingredients.vendorid** is a foreign key referencing **vendors.vendorid**.
- A foreign key depends on the row it is referencing in another table for completeness.
- In the ingredients table, a **vendorid** attribute value makes little sense without a corresponding row in the **vendors** table.
- The table containing the foreign key is called the **child table**, and the table containing the attribute(s) it references is called the **parent table**.
- The attribute(s) values referenced in the parent table are required to be unique. Usually, these referenced attributes are the primary keys of the parent table, but this is not a requirement.



# Foreign Key

- A foreign key (or referential) integrity constraint requires that any foreign key value in a child table must have one of the following characteristics:
  - Have a matching value in the parent table OR.
  - Be NULL.
- If we try to add a row for the ingredient eggplant with a vendor identifier of **FAKVD**, the DBMS will not allow the addition because the **vendorid** value **FAKVD** does not exist in the **vendors** table.

**ingredients**

| ingredientid | name            | vendorid |
|--------------|-----------------|----------|
| GRAPE        | Grape           | FRTFR    |
| LETUS        | Lettuce         | VGRUS    |
| PICKL        | Pickle          | VGRUS    |
| SCTDR        | Secret Dressing | NULL     |

**vendors**

| vendorid | companyname            | referredby |
|----------|------------------------|------------|
| FLVCR    | Flavorful Creams       | VGRUS      |
| FRTFR    | "Fruit Eating" Friends | FLVCR      |
| VGRUS    | Veggies_R_Us           | NULL       |

✗ Violates Foreign Key Constraint

|       |          |       |
|-------|----------|-------|
| EGPLT | Eggplant | FAKVD |
|-------|----------|-------|



## ① DML (Data Manipulation Language)



# Database Languages

- 1 DML (Data Manipulation Language)
- 2 DDL (Data Definition Language)



# Database Languages

- 1 DML (Data Manipulation Language)
- 2 DDL (Data Definition Language)
- 3 DCL (Data Control Language)



# Database Languages

- 1 DML (Data Manipulation Language)
- 2 DDL (Data Definition Language)
- 3 DCL (Data Control Language)
- 4 TCL (Transaction Control Language)



## Data Manipulation Language (DML)

- Enables users to access or manipulate data. The operations that can be performed include:
  - Retrieval of information stored in the database.
  - Insertion of new information.
  - Deletion of information.
  - Modification of information stored in the database.
- Types of data manipulation language:
  - **Procedural DML** specify what data are needed and how to get those data.
  - **Declarative DMLs** (also referred to as nonprocedural DMLs) specify what data are needed without specifying how to get those data.
- **Query** → A statement requesting the retrieval of information.
- The portion of DML that involves information retrieval is called **Data Query Language (DQL)**.



# Database Languages

- DML Commands:

- **SELECT**: retrieve the data from the database
- **INSERT**: insert into a table
- **UPDATE**: update the existing data within a table
- **DELETE**: Delete the records from a table

## Data Definition Language (DDL)

- It is used for defining the database schema.

Example:

```
create table account (account_number number(10), balance  
number(10,2));
```

- DDL compiler generates and stored schema description in a **data dictionary** or **catalog**.
- **Data Dictionary** is a special type of table consulted before any query or modification of the data.



# Database Languages

- DDL Commands:
  - **CREATE:** Create a database object like table, database.
  - **ALTER:** Alter existing table, column description.
  - **DROP:** Delete the existing object from the database.
  - **Rename:** Rename an existing database object

## Data Control Language (DCL)

- DCL commands are used to grant and take back authority from any database user.
- DCL Commands:
  - **GRANT:** Gives user's access privileges to the database.
  - **REVOKE:** Withdraw the access privileges from the user given with the Grant command



## Transaction Control Language (TCL)

- Transaction Control Language commands are used to manage transactions in the database.
- Used to manage the changes made by DML statements.
- TCL commands:
  - **COMMIT**: Permanently Save any update done into the database.
  - **SAVEPOINT**: Temporarily save any update so that you can later rollback to that point whenever necessary.
  - **ROLLBACK**: Restores the database to last committed state in case of failure. It is also used with savepoint command to jump to a savepoint in a transaction.



# Creating, Deleting and Altering Table (DDL)

- We can create a table in SQL using **CREATE TABLE**.

```
CREATE TABLE <table name> (  
    <column name> <type> [<default value>] [<column constraints>]  
    : : :  
    <column name> <type> [<default value>] [<column constraints>]  
    <table constraint>,  
    : : :  
    <table constraint>,  
);
```

- This creates a table named **<table name>**. The columns of the table are specified in a comma-delimited list of name/data type pairs. Let's create a simplified version of the **vendors** table.

```
CREATE TABLE vendors (  
    vendorid CHAR(5),  
    companyname VARCHAR(30),  
    repfname VARCHAR(20),  
    replname VARCHAR(20),  
    referredby CHAR(5)  
);
```



# Creating, Deleting and Altering Table (DDL)

## Default Value

- Any columns without a specified value are assigned the default value.
- Unless otherwise specified, the default value of a column is NULL.
- SQL allows you to specify a default value for a column using the DEFAULT clause.

DEFAULT <value expression>

- <value expression> is a simple expression of literals, functions, and values.
- <value expression> must evaluate a data type that matches or can be implicitly converted to the column data type.
- The DEFAULT clause follows the attribute name and type declaration.



# Creating, Deleting and Altering Table (DDL)

## Constraints

- DBMS can enforce rules (called **constraints**) on what data are allowed in the database. Such constraints are important because they help maintain data integrity.
- **For example**, you may want to ensure that each meal costs at least as much as its ingredients.
- When you specify the data type of a column, you constrain the possible values that the column may hold. This is called a **domain constraint**.
- **For example**, a column of type INTEGER may only hold whole numbers within a certain range. Any attempt to insert an invalid value will be rejected by SQL.
- SQL enforces constraints by prohibiting any data in the database that violates any constraint.



# Creating, Deleting and Altering Table (DDL)

- Any insert, update, or deletion that would result in a constraint violation is rejected without changing the database.
- There are two forms of constraint specification:
  - **Column constraint** → Declared with and applies to one particular column.
  - **Table constraint** → Declared separately and may apply to one or more columns



# Creating, Deleting and Altering Table (DDL)

## NOT NULL

- By default, most DBMSs allow NULL as a value for any column of any data type.
- Fortunately, you can require the database to prohibit NULL values for particular columns by using the NOT NULL column constraint.
- Many DBMSs also include a NULL column constraint, which specifies that NULL values are allowed; however, because this is the default behavior, this constraint usually is unnecessary.

```
CREATE TABLE vendors (  
    vendorid CHAR(5) NOT NULL,  
    companyname VARCHAR(30) DEFAULT 'SECRET' NOT NULL,  
    repfname VARCHAR(20) DEFAULT 'Mr./Ms.',  
    replname VARCHAR(20),  
    referredby CHAR(5) NULL  
);
```



# Creating, Deleting and Altering Table (DDL)

## UNIQUE

- The UNIQUE constraint forces distinct column values.

```
CREATE TABLE meals (  
    mealid CHAR(5) NOT NULL,  
    name CHAR(20) UNIQUE  
);
```

- Now the DBMS will not allow any row with a duplicate name value. Note that **UNIQUE** only applies to non-NULL values.
- What if we want to make the values for a set of columns unique?



# Creating, Deleting and Altering Table (DDL)

- For example, suppose we want to make sure that we don't have two vendor representatives with the same first and last name.

```
CREATE TABLE vendors (  
    vendorid CHAR(5) NOT NULL,  
    companyname VARCHAR(30) DEFAULT 'SECRET' NOT NULL,  
    repfname VARCHAR(20) DEFAULT 'Mr./Ms.' UNIQUE,  
    replname VARCHAR(20) UNIQUE,  
    referredby CHAR(5) NULL  
);
```

- Under these constraints, we could not have two representatives with the first name “Bob”, even if their last names differed, because the UNIQUE constraint for repfname only applies to that column.



# Creating, Deleting and Altering Table (DDL)

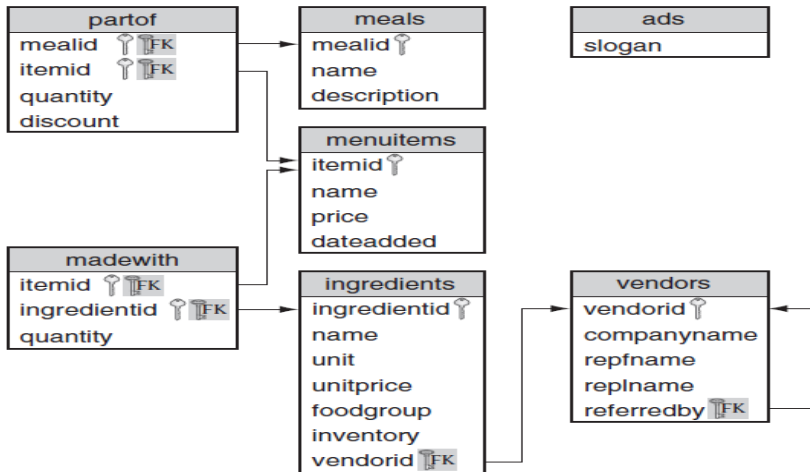
- Fortunately, UNIQUE also has a table constraint form that applies to the entire table instead of just a single column..

```
CREATE TABLE vendors (  
    vendorid CHAR(5) NOT NULL,  
    companyname VARCHAR(30) DEFAULT 'SECRET' UNIQUE,  
    repfname VARCHAR(20) DEFAULT 'Mr./Ms.',  
    replname VARCHAR(20),  
    referredby CHAR(5) NULL,  
    UNIQUE(repfname, replname)  
);
```

- The **UNIQUE(repfname, replname)** table constraint allows multiple vendor representatives with the same first or last name.



# Creating, Deleting and Altering Table (DDL)



## RESTURANT DATABASE SCHEMA



# Creating, Deleting and Altering Table (DDL)

**vendors**—Vendors who supply ingredients for the restaurant.

| Column                                                                                       | Type        | Description                                                                                            |
|----------------------------------------------------------------------------------------------|-------------|--------------------------------------------------------------------------------------------------------|
| vendorid    | CHAR(5)     | Unique vendor identifier                                                                               |
| companyname                                                                                  | VARCHAR(30) | Vendor company name                                                                                    |
| repfname                                                                                     | VARCHAR(20) | Vendor representative first name                                                                       |
| replname                                                                                     | VARCHAR(20) | Vendor representative last name                                                                        |
| referredby  | CHAR(5)     | Identifier of vendor that referred this vendor to you. Foreign key referencing <i>vendors.vendorid</i> |

**ingredients**—Ingredients provided by vendors for use in items.

| Column                                                                                         | Type         | Description                                                                                |
|------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------------|
| ingredientid  | CHAR(5)      | Unique ingredient identifier                                                               |
| name                                                                                           | VARCHAR(30)  | Ingredient name                                                                            |
| unit                                                                                           | CHAR(10)     | Ingredient serving size                                                                    |
| unitprice                                                                                      | NUMERIC(5,2) | Cost of an ingredient serving                                                              |
| foodgroup                                                                                      | CHAR(15)     | Ingredient food group                                                                      |
| inventory                                                                                      | INTEGER      | Number of available ingredient servings                                                    |
| vendorid      | CHAR(5)      | Identifier of vendor supplying ingredient. Foreign key referencing <i>vendors.vendorid</i> |



# Creating, Deleting and Altering Table (DDL)

**items**—Basic items for sale to customers.

| Column                                                                                   | Type         | Description             |
|------------------------------------------------------------------------------------------|--------------|-------------------------|
| itemid  | CHAR(5)      | Unique item identifier  |
| name                                                                                     | VARCHAR(30)  | Item name               |
| price                                                                                    | NUMERIC(5,2) | Item price              |
| dateadded                                                                                | DATE         | Date item added to menu |



# Creating, Deleting and Altering Table (DDL)

**madewith**—Item ingredients: Join table for the many-to-many relationship between *ingredients* and *items*.

| Column                                                                                                                                                                           | Type    | Description                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------------------------------------------------------------------------------------------------------|
| itemid         | CHAR(5) | Identifier of item that uses the ingredient. Foreign key referencing <i>items.itemid</i> .           |
| ingredientid   | CHAR(5) | Identifier of ingredient used in the item. Foreign key referencing <i>ingredients.ingredientid</i> . |
| quantity                                                                                                                                                                         | INTEGER | Number of units of specified ingredient to use in item.                                              |

**meals**—A meal is a collection of items. To entice customers to buy more food, you combine items together into a meal and sell it at a discount. For example, our Chicken N Suds meal combines a chicken salad with a soda.

| Column                                                                                   | Type     | Description            |
|------------------------------------------------------------------------------------------|----------|------------------------|
| mealid  | CHAR(5)  | Unique meal identifier |
| name                                                                                     | CHAR(10) | Meal name              |



# Creating, Deleting and Altering Table (DDL)

**partof**—Meal items: Join table for the many-to-many relationship between *meals* and *items*.

| Column                                                                                                                                                                     | Type         | Description                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mealid   | CHAR(5)      | Identifier of meal that includes the item. Foreign key referencing <i>meals.mealid</i> .                                                                                                                                     |
| itemid   | CHAR(5)      | Identifier of item to be used in the meal. Foreign key referencing <i>items.itemid</i> .                                                                                                                                     |
| quantity                                                                                                                                                                   | INTEGER      | Number of specified items to use in meal.                                                                                                                                                                                    |
| discount                                                                                                                                                                   | DECIMAL(2,2) | Percentage discount for item when part of this meal. To compute the price of a meal, sum the discounted individual price of all items. For example, <i>Chicken N Suds</i> meal costs<br>$2.85 * 98\% + 0.99 * 90\% = 3.68$ . |

**ads**—Possible advertising slogans for the restaurant.

| Column | Type     | Description        |
|--------|----------|--------------------|
| slogan | CHAR(50) | Advertising slogan |



# Creating, Deleting and Altering Table (DDL)

**menuitems**—Combination of both meals and items to go on the restaurant menu. We implement *menuitems* as a view (see Chapter 10).

| Column                                                                                       | Type         | Description             |
|----------------------------------------------------------------------------------------------|--------------|-------------------------|
| menuitemid  | CHAR(5)      | Item or meal identifier |
| name                                                                                         | VARCHAR(30)  | Menu item name          |
| price                                                                                        | NUMERIC(5,2) | Item price              |

**stores**—Stores franchised across the country.

| Column                                                                                    | Type        | Description                                                    |
|-------------------------------------------------------------------------------------------|-------------|----------------------------------------------------------------|
| storeid  | CHAR(5)     | Unique store identifier                                        |
| street                                                                                    | VARCHAR(25) | Street address of the store                                    |
| city                                                                                      | VARCHAR(25) | City of the store                                              |
| state                                                                                     | CHAR(2)     | U.S. state of the store                                        |
| zip                                                                                       | CHAR(10)    | U.S. zip code in XXXXX-XXXX format                             |
| operator                                                                                  | VARCHAR(41) | Name of the operator of the store (e.g., owner or the manager) |



# Creating, Deleting and Altering Table (DDL)

**orders**—Information about each order placed at every store. Each order that a customer places is given a unique order number. Each item within an order is given a line number, which is unique within the order. The *menuitemid* is not declared as a foreign key to allow for future deletion of menu items.

| Column                                                                                                                                                                      | Type         | Description                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------|
| ordernumber                                                                                | INTEGER      | Unique order identifier                                                                         |
| linenumber                                                                                 | INTEGER      | Order line identifier for orders with multiple items that is unique within an order             |
| storeid   | CHAR(5)      | Identifier of the store where the order is placed. Foreign key referencing <i>store.storeid</i> |
| menuitemid                                                                                                                                                                  | CHAR(5)      | Identifier of menu item ordered for this line number                                            |
| price                                                                                                                                                                       | NUMERIC(5,2) | Price of the menu item ordered                                                                  |
| time                                                                                                                                                                        | TIMESTAMP    | Time and date the order was placed                                                              |



# Creating, Deleting and Altering Table (DDL)

## PRIMARY KEY

- primary key of a table is a column or set of columns that uniquely identifies a row in the table.
- We can declare a primary key using the PRIMARY KEY constraint.

```
itemid CHAR(5) PRIMARY KEY
```

- For example, the primary key of the `partof` table contains two columns.
- You cannot simply add the PRIMARY KEY constraint to every column in the key because SQL will think you are trying to create multiple primary keys, which is not allowed.



# Creating, Deleting and Altering Table (DDL)

- Instead, use the table constraint form of PRIMARY KEY.

```
CREATE TABLE partof (  
    mealid CHAR(5),  
    itemid CHAR(5),  
    quantity INTEGER,  
    discount DECIMAL (2, 2) DEFAULT 0.00,  
    PRIMARY KEY(mealid, itemid)  
);
```

- Keep the following in mind when specifying the PRIMARY KEY constraint:
  - No values of the primary key columns may be NULL, so we don't need the NOT NULL constraint for **itemid**. In the **partof** table, neither **itemid** nor **ingredientid** may have a NULL value for any row.
  - Because the primary key uniquely identifies a row in the table, SQL will not allow two rows with the same value for all attributes of the primary key.



# Creating, Deleting and Altering Table (DDL)

- SQL allows tables to be created without a primary key; however, this is usually a bad idea because we won't be guaranteed to have a way to specify a particular row.
- The PRIMARY KEY and UNIQUE constraints differ in two ways:
  - UNIQUE allows NULL values for its columns.
  - There can be at most one PRIMARY KEY constraint for each table. There is no such limit on the number of UNIQUE constraints for a table.

## FOREIGN KEY

- A foreign key restricts the values of a column (or a set of columns) to the values appearing in another column (or set of columns) or to NULL.
- In table ingredients (child table), vendorid is a foreign key that refers to vendorid in table vendors (parent table).



# Creating, Deleting and Altering Table (DDL)

- In SQL, we specify a foreign key with the REFERENCES column constraint.

`REFERENCES <referenced table>[(<referenced column>)]`

- A column with a REFERENCES constraint may only have a value of either NULL or a value found in column `<referenced column>` of table `<referenced table>`.
- Here is the madewith table with the appropriate foreign keys.

```
CREATE TABLE madewith (  
    itemid CHAR(5) REFERENCES items(itemid),  
    ingredientid CHAR(5) REFERENCES ingredients,  
    quantity INTEGER DEFAULT 0 NOT NULL,  
    PRIMARY KEY(itemid, ingredientid)  
);
```

- With both the primary key and references constraints, the DBMS will only accept column values for itemid in madewith if the following are true:



# Creating, Deleting and Altering Table (DDL)

- The value exists in the items.itemid (foreign key constraint) AND
- The value is not NULL (primary key constraint)
- What about a foreign key with multiple columns? use the table constraint form of FOREIGN KEY.

```
FOREIGN KEY (<column list>)  
REFERENCES <referenced table>[(<referenced columns>)]
```

```
CREATE TABLE partof (  
    mealid CHAR(5),  
    itemid CHAR(5),  
    quantity INTEGER,  
    discount DECIMAL (2, 2) DEFAULT 0.00,  
    PRIMARY KEY(mealid, itemid),  
    FOREIGN KEY(mealid) REFERENCES meals(mealid),  
    FOREIGN KEY(itemid) REFERENCES items(itemid)  
);
```



# Creating, Deleting and Altering Table (DDL)

## CHECK

- A CHECK constraint specifies a boolean value expression to be evaluated for each row before allowing any data change.
- Any INSERT, UPDATE, or DELETE that would cause the condition for any row to evaluate to false is rejected by the DBMS.

**CHECK (<condition>)**

- A CHECK constraint may be specified as either a column or table constraint.
- In the following example, we specify two CHECK constraints on the ingredients table: a column CHECK constraint on the allowable foodgroup values and a table constraint over the inventory value of each ingredient.



# Creating, Deleting and Altering Table (DDL)

```
CREATE TABLE ingredients (  
    ingredientid CHAR(5) PRIMARY KEY,  
    name VARCHAR(30) NOT NULL,  
    unit CHAR(10),  
    unitprice NUMERIC(5,2),  
    foodgroup CHAR(15) CHECK (foodgroup IN ('Milk', 'Meat',  
    'Bread', 'Fruit', 'Vegetable')),  
    inventory INTEGER DEFAULT 0,  
    vendorid CHAR(5),  
    CHECK (unitprice * inventory <= 4000),  
    FOREIGN KEY(vendorid) REFERENCES vendors(vendorid)  
);
```



# Creating, Deleting and Altering Table (DDL)

## Drop Table

- You can remove a table using the following:

```
DROP TABLE <table name> [CASCADE | RESTRICT];
```

- DROP TABLE deletes the table < name > along with its data from the database.

```
DROP TABLE ads;
```

- Once you drop a table, it's gone, along with all of its data. You cannot undo a DROP TABLE.

```
DELETE FROM ads;
```

- The DELETE operation removes all of the rows from the table, but the table still remains.
- With DROP TABLE, the table itself is removed, along with all of its rows, all of its constraints, and everything else related to it.
- SQL does not allow a table to be dropped if its removal invalidates any constraints.



# Creating, Deleting and Altering Table (DDL)

- If you try to drop the vendors table, SQL will refuse because of the foreign key constraint on ingredients.vendorid.
- This is the default behavior or the result of using the RESTRICT option.
- To remove a table with constraints, we can specify CASCADE with DROP TABLE.
- This will cause any constraints referencing the table to be removed before the table is dropped.  
`DROP TABLE vendors CASCADE;`
- SQL begins by dropping the foreign key constraint from ingredients. Then, it can actually drop the vendors table.



# Creating, Deleting and Altering Table (DDL)

## Alter Table

- You can modify the columns and constraints of a table using the following:

```
ALTER TABLE <table name> <action>;
```

- There are several types of actions:

```
ADD [COLUMN] <column definition>
```

- Adds a new column to < *tablename* >. The <column definition> may contain any of the elements of a column definition in CREATE TABLE, such as types and constraints.

```
ALTER [COLUMN] <column name> [SET DEFAULT <value expression>
```

- Change or drop the specified default value from < *columnname* >.

```
DROP [COLUMN] <column name> [CASCADE | RESTRICT]
```



# Creating, Deleting and Altering Table (DDL)

- Delete `<column name>` from `<tablename>`. If there are any external dependencies on this column, the drop will be rejected. If *CASCADE* is specified, the DBMS will attempt to eliminate all external dependencies before deleting the column. If *RESTRICT* is specified, the DBMS will not allow the DROP if any external dependencies would be violated.

`ADD <table constraint>`

- Add a new constraint to `<tablename>`. `<tableconstraint>` uses the same syntax as table constraints in CREATE TABLE.

`DROP CONSTRAINT <constraint name> [CASCADE | RESTRICT]`

- Delete `<constraintname>` from `<tablename>`. If there are any external entities that depend on this constraint, the drop could invalidate the entity.



# Creating, Deleting and Altering Table (DDL)

- If *RESTRICT* is specified and an entity could be invalidated, the DBMS will not drop the constraint. If *CASCADE* is specified, the DBMS will attempt to eliminate all potentially violated entities before deleting the constraint.

```
ALTER TABLE vendors
```

```
ADD COLUMN url VARCHAR(100) DEFAULT 'unlisted';
```

- The new attribute, url, is added to the table. The url value for all of the existing rows is unlisted. If we hadn't specified a default, all of the existing rows would have a value of NULL for the url attribute.
- Rename a column:

```
ALTER TABLE VENDORS
```

```
RENAME COLUMN COMPANYNAME TO CNAME;
```

- Modify Column datatype:

```
ALTER TABLE VENDORS
```

```
MODIFY CNAME VARCHAR(10);
```



# Modifying Data

## INSERT: Adding New Rows

- The INSERT statement adds new rows to a specified table.
- There are two variants of the INSERT statement. One inserts a single row of values into the database, whereas the other inserts multiple rows returned from a SELECT.

## INSERTing a Row with VALUES

- The most basic form of INSERT creates a single, new row with either user-specified or default values.
- The syntax of this INSERT statement is as follows:  

```
INSERT INTO <table name>  
[(<attribute>, : : :, <attribute>)]  
VALUES (<expression>, : : :, <expression>);
```
- INSERT creates a new row in < *tablename* >. The new row contains the values determined by the expressions in the VALUES list.



# Modifying Data

- The expressions in the VALUES list correspond to the attributes in the comma separated following the table name. If no attributes are given, then the order of the attributes in  $\langle \text{tablename} \rangle$  is used.

```
INSERT INTO ingredients VALUES  
( 'EDSDR', 'Ed's Vanishing Dressing', 'ounce',  
0.1, NULL, DEFAULT, 'EDDRS');
```

- What happens if somebody changes the order or number of attributes in the ingredients table? SQL may refuse to execute our INSERT statement.
- Even worse, if only the order of the attributes changes, SQL may put values in the wrong place without any warning.



# Modifying Data

- In general, it is good to avoid this reliance on the attribute creation order by naming the attributes ourselves as shown below:

```
INSERT INTO ingredients
(ingredientid, name, unitprice, unit, foodgroup, vendorid)
VALUES ('EDSDR', 'Ed''s Vanishing Dressing', 0.1,
'ounce', NULL, 'EDDRS');
```

- There is a special form of INSERT that creates a new row with all default values. You don't have to specify any data.

```
INSERT INTO ads DEFAULT VALUES;
```

## DELETE: Removing Rows

- You can remove rows from a table using DELETE.

```
DELETE FROM <table name> [[AS] <alias>] [WHERE <condition>]
```

- DELETE removes all rows from < *tablename* > where < *condition* > evaluates to true.



# Modifying Data

- The WHERE is optional. If there is no WHERE clause, DELETE removes all rows.
- DELETE can only remove rows from a single table.

```
DELETE FROM items WHERE price IS NULL;
```

## UPDATE: Changing Row Values

- You can change the values in existing rows using UPDATE.

```
UPDATE <table-name> [[AS] <alias>]  
SET <column>=<expression>, : : :, <attribute>=<expression>  
[WHERE <condition>];
```

- UPDATE changes all rows in < *tablename* > where < *condition* > evaluates to true.
- For each row, the SET clause dictates which attributes change and how to compute the new value. All other attribute values do not change.



# Modifying Data

- The WHERE is optional, but Like DELETE, if there is no WHERE clause, UPDATE changes all rows.

- UPDATE can only change rows from a single table.

```
UPDATE ingredients  
SET inventory = 0  
WHERE foodgroup = 'Milk';
```

- UPDATE goes row-by-row through ingredients changing the inventory attribute value to 0 for any row satisfying foodgroup = 'Milk'.
- Whereas the updates can only apply to one table, the WHERE clause can involve any number of tables.



# Modifying Data

## vendors

| vendorid  | companyname            | repfname | replname | referredby  |
|--------------------------------------------------------------------------------------------|------------------------|----------|----------|------------------------------------------------------------------------------------------------|
| VGRUS                                                                                      | Veggies_R_Us           | Candy    | Corn     | NULL                                                                                           |
| DNDRY                                                                                      | Don's Dairy            | Marla    | Milker   | VGRUS                                                                                          |
| FLVCR                                                                                      | Flavorful Creams       | Sherman  | Sherbert | VGRUS                                                                                          |
| FRTFR                                                                                      | "Fruit Eating" Friends | Gilbert  | Grape    | FLVCR                                                                                          |
| EDDRS                                                                                      | Ed's Dressings         | Sam      | Sauce    | FRTFR                                                                                          |
| SPWTR                                                                                      | Spring Water Supply    | Gus      | Hing     | EDDRS                                                                                          |

## ingredients

| ingredientid  | name            | unit  | unitprice | foodgroup | inventory | vendorid  |
|------------------------------------------------------------------------------------------------|-----------------|-------|-----------|-----------|-----------|----------------------------------------------------------------------------------------------|
| CHESE                                                                                          | Cheese          | scoop | 0.03      | Milk      | 150       | DNDRY                                                                                        |
| CHIKN                                                                                          | Chicken         | strip | 0.45      | Meat      | 120       | DNDRY                                                                                        |
| CRUTN                                                                                          | Crouton         | piece | 0.01      | Bread     | 400       | EDDRS                                                                                        |
| GRAPE                                                                                          | Grape           | piece | 0.01      | Fruit     | 300       | FRTFR                                                                                        |
| LETUS                                                                                          | Lettuce         | bowl  | 0.01      | Vegetable | 200       | VGRUS                                                                                        |
| PICKL                                                                                          | Pickle          | slice | 0.04      | Vegetable | 800       | VGRUS                                                                                        |
| SCTDR                                                                                          | Secret Dressing | ounce | 0.03      | NULL      | 120       | NULL                                                                                         |
| TOMTO                                                                                          | Tomato          | slice | 0.03      | Fruit     | 15        | VGRUS                                                                                        |
| WATER                                                                                          | Water           | glass | 0.06      | NULL      | NULL      | SPWTR                                                                                        |
| SODA                                                                                           | Soda            | glass | 0.69      | NULL      | 5000      | SPWTR                                                                                        |
| WTRML                                                                                          | Watermelon      | piece | 0.02      | Fruit     | NULL      | FRTFR                                                                                        |
| ORNG                                                                                           | Orange          | slice | 0.05      | Fruit     | 10        | FRTFR                                                                                        |



# Modifying Data

items

| itemid  | name             | price | dateadded  |
|------------------------------------------------------------------------------------------|------------------|-------|------------|
| CHKSD                                                                                    | Chicken Salad    | 2.85  | 1998-11-13 |
| FRTSD                                                                                    | Fruit Salad      | 3.45  | 2000-05-06 |
| GDNSD                                                                                    | Garden Salad     | 0.99  | 2001-03-02 |
| MILSD                                                                                    | Millennium Salad | NULL  | 2002-08-16 |
| SODA                                                                                     | Soda             | 0.99  | 2003-02-06 |
| WATER                                                                                    | Water            | 0.00  | 2002-05-19 |
| FRPLT                                                                                    | Fruit Plate      | 3.99  | 2000-09-02 |

meals

| mealid  | name           |
|------------------------------------------------------------------------------------------|----------------|
| CKSDS                                                                                    | Chicken N Suds |
| VGNET                                                                                    | Vegan Eatin'   |

partof

| mealid  | itemid  | quantity | discount |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------|----------|
| CKSDS                                                                                    | CHKSD                                                                                    | 1        | 0.02     |
| CKSDS                                                                                    | SODA                                                                                     | 1        | 0.10     |
| VGNET                                                                                    | GDNSD                                                                                    | 1        | 0.03     |
| VGNET                                                                                    | FRTSD                                                                                    | 1        | 0.01     |
| VGNET                                                                                    | WATER                                                                                    | 1        | 0.00     |

madewith

| itemid  | ingredientid  | quantity |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------|
| CHKSD                                                                                    | CHESE                                                                                            | 2        |
| CHKSD                                                                                    | CHIKN                                                                                            | 4        |
| CHKSD                                                                                    | LETUS                                                                                            | 1        |
| CHKSD                                                                                    | SCTDR                                                                                            | 1        |
| FRTSD                                                                                    | GRAPE                                                                                            | 10       |
| FRTSD                                                                                    | WTRML                                                                                            | 5        |
| GDNSD                                                                                    | LETUS                                                                                            | 4        |
| GDNSD                                                                                    | TOMTO                                                                                            | 8        |
| FRPLT                                                                                    | WTRML                                                                                            | 10       |
| FRPLT                                                                                    | GRAPE                                                                                            | 10       |
| FRPLT                                                                                    | CHESE                                                                                            | 10       |
| FRPLT                                                                                    | CRUTN                                                                                            | 10       |
| FRPLT                                                                                    | TOMTO                                                                                            | 8        |
| WATER                                                                                    | WATER                                                                                            | 1        |
| SODA                                                                                     | SODA                                                                                             | 1        |
| FRPLT                                                                                    | ORNG                                                                                             | 10       |

ads

| slogan                          |
|---------------------------------|
| Grazing in style                |
| NULL                            |
| Bovine friendly and heart smart |
| Where the grazin's good         |
| The grass is greener here       |
| Welcome to the "other side"     |



# Modifying Data

menuitems

| menuitemid | name             | price |
|------------|------------------|-------|
| CHKSD      | Chicken Salad    | 2.85  |
| CKSDS      | Chicken N Suds   | 3.68  |
| FRPLT      | Fruit Plate      | 3.99  |
| FRTSD      | Fruit Salad      | 3.45  |
| GDNSD      | Garden Salad     | 0.99  |
| MILSD      | Millennium Salad | NULL  |
| SODA       | Soda             | 0.99  |
| VGNET      | Vegan Eatin'     | 4.38  |
| WATER      | Water            | 0.00  |

stores

| storeid | address       | city          | state | zip        | manager      |
|---------|---------------|---------------|-------|------------|--------------|
| FIRST   | 1111 Main St. | Waco          | TX    | 76798      | Jeff Donahoo |
| #2STR   | 2222 2nd Ave. | Waco          | TX    | 76798-7356 | Greg Speegle |
| NDSTR   | 3333 3rd St.  | Fargo         | ND    | 58106      | Jeff Speegle |
| CASTR   | 4444 4th Blvd | San Francisco | CA    | 94101-4150 | Greg Donahoo |
| NWSTR   | NULL          | NULL          | TX    | NULL       | Man Ager     |

orders

| ordernumber | linenumber | storeid | menuitemid | price | time                    |
|-------------|------------|---------|------------|-------|-------------------------|
| 1           | 1          | FIRST   | FRTSD      | 3.45  | 2005-01-26 13:46:04.188 |
| 1           | 2          | FIRST   | WATER      | 0.00  | 2005-01-26 13:46:19.188 |
| 1           | 3          | FIRST   | WATER      | 0.00  | 2005-01-26 13:46:34.188 |
| 2           | 1          | FIRST   | CHKSD      | 2.85  | 2005-01-26 13:47:49.188 |
| 3           | 1          | FIRST   | SODA       | 0.99  | 2005-01-26 13:49:04.188 |
| 3           | 2          | FIRST   | FRPLT      | 3.99  | 2005-01-26 13:49:19.188 |
| 3           | 3          | FIRST   | VGNET      | 4.38  | 2005-01-26 13:49:34.188 |
| 1           | 1          | #2STR   | CKSDS      | 3.68  | 2005-01-26 14:02:04.188 |
| 1           | 2          | #2STR   | CHKSD      | 2.85  | 2005-01-26 14:02:19.188 |
| 1           | 3          | #2STR   | SODA       | 0.99  | 2005-01-26 14:02:34.188 |
| 1           | 4          | #2STR   | GDNSD      | 0.99  | 2005-01-26 14:02:49.188 |
| 2           | 1          | #2STR   | CHKSD      | 2.85  | 2005-01-26 14:04:04.188 |
| 2           | 2          | #2STR   | SODA       | 0.99  | 2005-01-26 14:04:19.188 |
| 3           | 1          | #2STR   | CHKSD      | 2.85  | 2005-01-26 14:05:34.188 |
| 3           | 2          | #2STR   | FRPLT      | 3.99  | 2005-01-26 14:05:49.188 |
| 3           | 3          | #2STR   | GDNSD      | 0.99  | 2005-01-26 14:06:04.188 |
| 1           | 1          | NDSTR   | WATER      | 0.00  | 2005-01-26 14:14:04.188 |
| 1           | 2          | NDSTR   | FRPLT      | 3.99  | 2005-01-26 14:14:19.188 |



# Retrieval: Basic SELECTION

- We only need to tell SELECT two things:
  - 1) the data attributes you want and
  - 2) the table where it should get it.
- Suppose you wanted to find the names of all individual items on your menu.

```
SELECT name FROM items;
```

- To retrieve multiple columns, specify a comma-delimited list of column names.

```
SELECT name, itemid, price FROM items;
```

- What if we want all of the columns in a table? SQL provides \* as a shorthand for all columns.

```
SELECT * FROM items;
```



# Comments

SQL has two types of comments. The first type begins with two minus signs ('-') and ends with the end of the line. The second type begins with /\*, ends with \*/, and can span multiple lines.

```
/* This is a multiline comment */
```

```
SELECT itemid from items; -- This comment goes to the end of line
```



# Selecting rows

- Using basic SELECT/FROM, we can retrieve data from any table and even specify columns. But what if we want to limit the rows in the result?
- SQL uses the WHERE clause to specify the condition that a particular row must satisfy to be in the query result. SQL provides many different operators for condition construction.

| Operator      | Evaluates                                      | Usage                                   |
|---------------|------------------------------------------------|-----------------------------------------|
| =             | Equal                                          | <i>repfname</i> = 'Bob'                 |
| >, <          | Greater/less than                              | <i>unitprice</i> > 10                   |
| >=, <=        | Greater/less than or equal to                  | <i>inventory</i> <= 5000                |
| <>            | Not equal                                      | <i>vendorid</i> <> 3                    |
| [NOT] BETWEEN | Between two values (inclusive)                 | <i>vendorid</i> BETWEEN 'A' AND 'M'     |
| IS [NOT] NULL | Value is NULL                                  | <i>referredby</i> IS NULL               |
| [NOT] LIKE    | Equal strings using wildcards (e.g., '%', '_') | <i>replname</i> LIKE '%ith'             |
| [NOT] IN      | Equal to any element in a list                 | <i>name</i> IN ('Soda', 'Water')        |
| NOT           | Negates a condition                            | NOT <i>itemid</i> IN ('GDNSD', 'CHKSD') |



# Selecting rows

- Few Examples of standard comparison operators:

```
SELECT name FROM items WHERE price <= 0.99;
```

```
SELECT ingredientid, foodgroup FROM ingredients  
WHERE name = 'Cheese';
```

```
SELECT * FROM items WHERE dateadded > '1999-12-31';
```

## String Comparisons

- Find all items with a name less than or equal to 'garden'

```
SELECT name FROM items WHERE name <= 'garden';
```

- Find the name of the representative for Don's Dairy

```
SELECT repfname, replname FROM vendors  
WHERE companyname = 'Don"s Dairy';
```



# Selecting rows

## Matching String Patterns with LIKE

- SQL provides wildcard-based, pattern-matching capabilities with the LIKE operator. SQL wildcard characters include the following:

| Wildcard | Description                                           |
|----------|-------------------------------------------------------|
| %        | matches any substring containing 0 or more characters |
| _        | matches any single character                          |

- Find the list of vendor representative first names that begin with 'S'  
`SELECT repfname FROM vendors WHERE repfname LIKE 'S%';`
- Let's try a few examples using LIKE. Suppose you have the following list of character strings: Ball, Beggy, Bill, Billy, ill, Meg, Phill, Philly, Peg, Peggy.



# Selecting rows

- Following are some example LIKE expressions along with what they match:

| LIKE   | Matches strings                                                                           | Matches          |
|--------|-------------------------------------------------------------------------------------------|------------------|
| %ill   | ending in 'ill'                                                                           | Bill, ill, Phill |
| Peg%   | starting with 'Peg'                                                                       | Peg, Peggy       |
| B%y    | starting with B and ending with y                                                         | Beggy, Billy     |
| _ill   | starting with any single character and ending in 'ill'                                    | Bill             |
| B_ll   | starting with B and ending with 'll' with any single character in between                 | Ball, Bill       |
| %_ill_ | starting with 2 or more characters and ending with 'ill' followed by any single character | Philly           |

- What about using LIKE to find strings containing wildcard characters? For example, what if we wanted to find all strings containing an underscore character?



# Selecting rows

- We could try LIKE '%\_%', but this finds all strings containing 1 or more characters because SQL treats '\_' as a wildcard character. We need some way to turn a wildcard into a regular character.

- Find all vendor names containing an '\_'

```
SELECT companyname FROM vendors
```

```
WHERE companyname LIKE '%#_%' ESCAPE '#';
```

- Because '#' is designated as the escape character, '\_' is interpreted literally, not as a wildcard.
- To fetch the rows that evaluate to false, prefix the condition with NOT.
- Find the name of all of the food items other than salads

```
SELECT name FROM items WHERE NOT name LIKE '%Salad';
```



# Selecting rows

## Combining Conditions with AND and OR

- We can combine two or more conditions using AND and OR. Two conditions joined by AND must both be true for the entire condition to evaluate to true.

- Find all of the ingredients from the Fruit food group with an inventory greater than 100

```
SELECT ingredientid, name FROM ingredients  
WHERE foodgroup = 'Fruit' AND inventory > 100;
```

- For two conditions joined by OR, the entire condition is true if either or both are true.

- Find the name of all ingredients with unit price over \$0.40 or with a unit of glass.

```
SELECT name, unitprice, unit FROM ingredients  
WHERE unitprice > 0.40 OR unit = 'glass';
```



# Selecting rows

- You can combine as many ANDs and ORs together as needed, but beware. SQL may not interpret the WHERE clause as you intended.
- Some operators have a higher precedence than others. Operators with a higher precedence get evaluated before operators with a lower precedence, no matter where they are in the condition.
- In SQL, the precedence order from highest to lowest is as follows:

| Precedence | Operator             |
|------------|----------------------|
| Highest    | Comparison Operators |
|            | NOT                  |
|            | AND                  |
| Lowest     | OR                   |

- The evaluation of a condition begins with the individual comparisons.
- INCORRECT!** Find the food items that have a name beginning with either F or S that cost less than \$3.50.

```
SELECT name, price FROM items
```

```
WHERE name LIKE 'F%' OR name LIKE 'S%' AND price < 3.50;
```



# Selecting rows

- **CORRECT!** Find the food items that have a name beginning with either F or S that cost less than \$3.50.

```
SELECT name, price FROM items
```

```
WHERE (name LIKE 'F%' OR name LIKE 'S%') AND price < 3.50;
```

## Selecting a Range of Values with BETWEEN

- The BETWEEN operator allows us to specify a range of values (inclusive) for matching.
- Find the food items costing between \$2.50 and \$3.50

```
SELECT * FROM items WHERE price BETWEEN 2.50 AND 3.50;
```

- To get the values that are not within a specified range, use NOT BETWEEN.

- Find the food items costing less than \$2.50 or more than \$3.50

```
SELECT * FROM items WHERE price NOT BETWEEN 2.50 AND 3.50;
```



# Selecting rows

## Selecting a Set of Values Using IN

- The IN operator determines if a value is contained in a list of values.
- Find the vendor representatives with last names of Corn or Sherbert  
`SELECT vendorid, repfname, replname FROM vendors  
WHERE replname IN ('Corn', 'Sherbert');`

- you can use NOT IN to find all of the values not contained in a list.
- Find the ingredient ID, name, and unit of items not sold in pieces or strips.

```
SELECT ingredientid, name, unit FROM ingredients  
WHERE unit NOT IN ('piece', 'strip');
```

## IS NULL: Exploring the Unknown

- How does SQL handle unknown? SQL only reports the rows for which the WHERE condition evaluates to true. Rows evaluating to false or unknown are not included in the answer.



# Selecting rows

- **INCORRECT!** Find all vendors not referred by anyone  
`SELECT * FROM vendors WHERE referredby = NULL;`
- **CORRECT!** Find all vendors not referred by anyone  
`SELECT * FROM vendors WHERE referredby IS NULL;`  
ANDs, ORs, NOTs with NULLs: Three-Valued Logic
- What happens when we AND, OR, or NOT a value of unknown? SQL evaluates conditions using three-valued logic.

| AND     | true    | false   | unknown |
|---------|---------|---------|---------|
| true    | true    | false   | unknown |
| false   | false   | false   | false   |
| unknown | unknown | false   | unknown |
| OR      | true    | false   | unknown |
| true    | true    | true    | true    |
| false   | true    | false   | unknown |
| unknown | true    | unknown | unknown |
| NOT     | true    | false   | unknown |
| true    | false   |         |         |
| false   | true    |         |         |
| unknown | unknown |         |         |



# Selecting rows

- Let's try our three-valued logic tables on an example.
- Find the ingredient ID, food group, and inventory for fruits or ingredients with inventory not less than or equal to 200.

```
SELECT ingredientid, foodgroup, inventory FROM ingredients  
WHERE foodgroup = 'Fruit' OR NOT inventory <= 200;
```

| ingredientid | foodgroup | inventory |
|--------------|-----------|-----------|
| CRUTN        | Bread     | 400       |
| GRAPE        | Fruit     | 300       |
| PICKL        | Vegetable | 800       |
| TOMTO        | Fruit     | 15        |
| SODA         | NULL      | 5000      |
| WTRML        | Fruit     | NULL      |
| ORNG         | Fruit     | 10        |

- Let's consider two other rows in the ingredients table: the ones identified by WTRML and SCTDR. First, let's evaluate the condition for the WTRML row from ingredients:



## Selecting rows

$$\left. \begin{array}{l} \text{foodgroup} = \text{'Fruit'} = \text{true} \\ \text{NOT inventory} \leq 200 = \text{NOT unknown} = \text{unknown} \end{array} \right\} \text{OR} = \text{true}$$

- The first condition,  $\text{foodgroup} = \text{'Fruit'}$ , evaluates to true.  $\text{inventory} \leq 200$  evaluates to unknown, and the NOT of unknown is itself unknown.
- For this row, SQL evaluates true OR unknown, which is true so the WTRML row is included.
- Now consider the SCTDR row.

$$\left. \begin{array}{l} \text{foodgroup} = \text{'Fruit'} = \text{unknown} \\ \text{NOT inventory} \leq 200 = \text{NOT true} = \text{false} \end{array} \right\} \text{OR} = \text{unknown}$$

- $\text{foodgroup} = \text{'Fruit'}$  evaluates to unknown, and  $\text{NOT inventory} \leq 200$  evaluates to false. Because false OR unknown is unknown, the SCTDR row is not included in the results.



# Reshaping Results

## AS: Naming Result Table Columns

- We assign a column alias in the column list of the SELECT statement using AS.
- The AS keyword is optional.

```
SELECT companyname AS company, repfname AS "First Name"  
FROM vendors;
```

## DISTINCT and ALL: Dealing with Duplicates

- Duplicate rows are okay in SQL; however, you may not be so happy with all of that repeatedly repeating repetition.
- Find the food groups served by your restaurant without duplication  

```
SELECT DISTINCT foodgroup FROM ingredients;
```
- Note that NULL is treated as a distinct value, and duplicate NULLs are eliminated.



# Reshaping Results

- Find the distinct list of food groups provided by each vendor

```
SELECT DISTINCT foodgroup, vendorid  
FROM ingredients;
```

## Derived Attributes

- Numeric:** SQL can evaluate simple arithmetic expressions containing numeric columns and literals.
- Find the value of your pickle inventory if you double your stock of pickles  

```
SELECT ingredientid,  
inventory * 2 * unitprice AS "Inventory Value"  
FROM ingredients WHERE name = 'Pickle';
```
- SQL also includes many standard mathematic functions. The exact set of available functions is DBMS dependent.



# Reshaping Results

| Function     | Returns                       | Example               |
|--------------|-------------------------------|-----------------------|
| ABS(N)       | Absolute value of N           | ABS(inventory - 100)  |
| CEIL[ING](N) | Ceiling of N                  | CEILING(inventory/10) |
| EXP(N)       | $e^N$                         | EXP(5)                |
| FLOOR(N)     | Floor of N                    | FLOOR(inventory/10)   |
| LN(N)        | Natural log of N              | LN(5)                 |
| MOD(N, D)    | Remainder of N divided by D   | MOD(11, 3)            |
| POWER(B, E)  | B to the power of E ( $B^E$ ) | POWER(2, 3)           |
| SQRT(N)      | $\sqrt{N}$                    | SQRT(4)               |

## SQL Arithmetic Functions

```
SELECT name, inventory * unitprice AS Dollars,  
       CEIL(inventory * unitprice * 1.2552) AS euros,  
       1.2552 AS "Exchange Rate"  
FROM ingredients;
```



# Reshaping Results

| name            | dollars | euros | Exchange Rate |
|-----------------|---------|-------|---------------|
| Cheese          | 4.50    | 6     | 1.2552        |
| Chicken         | 54.00   | 68    | 1.2552        |
| Crouton         | 4.00    | 6     | 1.2552        |
| Grape           | 3.00    | 4     | 1.2552        |
| Lettuce         | 2.00    | 3     | 1.2552        |
| Pickle          | 32.00   | 41    | 1.2552        |
| Secret Dressing | 3.60    | 5     | 1.2552        |
| Tomato          | 0.45    | 1     | 1.2552        |
| Water           | NULL    | NULL  | 1.2552        |
| Soda            | 3450.00 | 4331  | 1.2552        |
| Watermelon      | NULL    | NULL  | 1.2552        |
| Orange          | 0.50    | 1     | 1.2552        |

- **String Concatenation:** We begin with the `||` (concatenation) operator to construct a new string from a combination of string expressions.
- Create a mailing label for each store

```
SELECT manager, address || ' ' || city || ' ' || state  
|| ' ' || zip || ' USA'  
as mail FROM stores;
```



# Reshaping Results

| manager      | mail                                          |
|--------------|-----------------------------------------------|
| Greg Speegle | 2222 2nd Ave. Waco TX 76798-7356 USA          |
| Greg Donahoo | 4444 4th Blvd San Francisco CA 94101-4150 USA |
| Jeff Donahoo | 1111 Main St. Waco TX 76798 USA               |
| Jeff Speegle | 3333 3rd St. Fargo ND 58106 USA               |
| Man Ager     | NULL                                          |

- You may use string literals in the SELECT list either by themselves or in concatenations.
- The concatenation operator can take both literals and columns. In fact, it takes any expression that returns (or can be coerced to) a string.
- Concatenation with NULL is always NULL.
- String concatenation does not add a space. Any spacing must be explicitly added.



# Reshaping Results

- Because strings of type CHAR are padded with trailing blanks, concatenation with such strings includes these extra blanks. See the spaces between the zip code and USA in some of the rows.
- VARCHARs are not padded with trailing spaces; they only include the explicitly specified characters.

- SUBSTRING:**

`SUBSTRING(<source> FROM <start> [FOR <length>])`

- SUBSTRING returns the substring from <source> string, starting from the character at position <start> (numbering from 1) and containing up to <length> characters.

`SELECT SUBSTRING(repfname FROM 1 FOR 1) || '. ' || replname  
AS name FROM vendors;`

| name        |
|-------------|
| C. Corn     |
| M. Milker   |
| S. Sherbert |
| G. Grape    |
| S. Sauce    |
| G. Hing     |



# Reshaping Results

- **TRIM:**

```
SELECT DISTINCT ingredientid, foodgroup || ' '
AS "with trailing", TRIM(TRAILING ' ' FROM foodgroup) ||
' ' AS "without trailing" FROM ingredients
WHERE inventory > 500;
```

| ingredientid | with trailing | without trailing |
|--------------|---------------|------------------|
| PICKL        | Vegetable     | Vegetable        |
| SODA         | NULL          | NULL             |

- **UPPER and LOWER:**

```
SELECT UPPER(repfname || ' ' || replname) AS rep,
LOWER(companyname) AS company FROM vendors
WHERE referredby = 'VGRUS';
```

- **POSITION: Finding Where a Substring Begins:**

```
SELECT name, POSITION('Salad' IN name) FROM items;
```



# Reshaping Results

| name             | position |
|------------------|----------|
| Chicken Salad    | 9        |
| Fruit Salad      | 7        |
| Garden Salad     | 8        |
| Millennium Salad | 12       |
| Soda             | 0        |
| Water            | 0        |
| Fruit Plate      | 0        |

- CHARACTER\_LENGTH: Counting Characters in a String:

```
SELECT name, CHARACTER_LENGTH(name) AS namelen,  
       CHARACTER_LENGTH(foodgroup) AS fglen  
FROM ingredients;
```



# Reshaping Results

| name            | namelen | fglen |
|-----------------|---------|-------|
| Cheese          | 6       | 15    |
| Chicken         | 7       | 15    |
| Crouton         | 7       | 15    |
| Grape           | 5       | 15    |
| Lettuce         | 7       | 15    |
| Pickle          | 6       | 15    |
| Secret Dressing | 15      | NULL  |
| Tomato          | 6       | 15    |
| Water           | 5       | NULL  |
| Soda            | 4       | NULL  |
| Watermelon      | 10      | 15    |
| Orange          | 6       | 15    |



# Reshaping Results

- Temporal: Finding the Current Date or Time

| Temporal Function              | Description                                      | Return Type                |
|--------------------------------|--------------------------------------------------|----------------------------|
| CURRENT_TIME[(precision)]      | Current time with time zone displacement         | TIME WITH TIMEZONE         |
| CURRENT_DATE                   | Current date                                     | DATE                       |
| CURRENT_TIMESTAMP[(precision)] | Current date and time with timezone displacement | TIMESTAMP WITH TIMEZONE    |
| LOCALTIME[(precision)]         | Current time                                     | TIME WITHOUT TIMEZONE      |
| LOCALTIMESTAMP[(precision)]    | Current date                                     | TIMESTAMP WITHOUT TIMEZONE |

- Find how long each item has been on the menu as of midnight January 2, 2005

```
SELECT name, dateadded,  
DATE '2005-01-02' - dateadded AS "Days on Menu"  
FROM items;
```



# Reshaping Results

| name             | dateadded  | Days on Menu |
|------------------|------------|--------------|
| Chicken Salad    | 1998-11-13 | 2242         |
| Fruit Salad      | 2000-05-06 | 1702         |
| Garden Salad     | 2001-03-02 | 1402         |
| Millennium Salad | 2002-08-16 | 870          |
| Soda             | 2003-02-06 | 696          |
| Water            | 2002-05-19 | 959          |
| Fruit Plate      | 2000-09-02 | 1583         |

- **EXTRACT: Getting Fields from Temporal Data**

```
SELECT name, EXTRACT(YEAR FROM dateadded) AS year,  
EXTRACT(MONTH FROM dateadded + INTERVAL '30' DAY) AS month  
FROM items;
```

| name             | year | month |
|------------------|------|-------|
| Chicken Salad    | 1998 | 12    |
| Fruit Salad      | 2000 | 6     |
| Garden Salad     | 2001 | 4     |
| Millennium Salad | 2002 | 9     |
| Soda             | 2003 | 3     |
| Water            | 2002 | 6     |
| Fruit Plate      | 2000 | 10    |



# Reshaping Results

- ORDER BY: Ordering Result Table Traversal
- Find all items from least to most expensive

```
SELECT name, price FROM items  
ORDER BY price ASC;
```

| name             | price |
|------------------|-------|
| Water            | 0.00  |
| Garden Salad     | 0.99  |
| Soda             | 0.99  |
| Chicken Salad    | 2.85  |
| Fruit Salad      | 3.45  |
| Fruit Plate      | 3.99  |
| Millennium Salad | NULL  |

- Find items added in 2001 or later in decreasing order of price

```
SELECT itemid, price FROM items  
WHERE EXTRACT(YEAR FROM dateadded) >= 2001  
ORDER BY price DESC;
```



# Reshaping Results

- Find the order number, line number, and item ID ordered at #2STR, last order first

```
SELECT ordernumber, linenummer, menuitemid FROM orders  
WHERE storeid = '#2STR'  
ORDER BY ordernumber DESC, linenummer ASC;
```

- CAST: Data Type Conversion

- In SQL, all data have a type, whether it is data in a table or data returned by some expression. Sometimes the data type needs to change.

```
SELECT name || ' was added on ' ||  
dateadded || ' and is ' || price AS message  
FROM items;
```

- Here the dateadded and price fields must be converted to character strings for concatenation.



# Reshaping Results

- In many cases, the DBMS silently converts data types. Such conversions are called **implicit type conversions**.
- In some cases, the DBMS needs help in determining the correct data type. To handle this, SQL provides the CAST operator, which allows the specification of the resulting data type.

```
SELECT CAST(name AS CHAR(20)) || '$' ||  
CAST(price AS NUMERIC(5,2)) AS "Menu"  
FROM menuitems  
WHERE CAST(menuitemid AS CHAR(1)) IN ('S', 'C', 'V');
```

| Menu           |        |
|----------------|--------|
| Chicken Salad  | \$2.85 |
| Chicken N Suds | \$3.68 |
| Soda           | \$0.99 |
| Vegan Eatin'   | \$4.38 |



# Reshaping Results

- CASE: Value List
- Consider the case where you want to list ingredients and their corresponding goodness as determined by the food group.
- The goodness assignments are as follows: Vegetable and Fruit are Good, Milk and Bread are Acceptable, Meat is Bad, and any other value is NULL, representing an unknown value.

```
SELECT name,  
       CASE foodgroup  
         WHEN 'Vegetable' THEN 'Good'  
         WHEN 'Fruit' THEN 'Good'  
         WHEN 'Milk' THEN 'Acceptable'  
         WHEN 'Bread' THEN 'Acceptable'  
         WHEN 'Meat' THEN 'Bad'  
       END AS quality  
FROM ingredients;
```



# Reshaping Results

| <b>name</b>     | <b>quality</b> |
|-----------------|----------------|
| Cheese          | Acceptable     |
| Chicken         | Bad            |
| Crouton         | Acceptable     |
| Grape           | Good           |
| Lettuce         | Good           |
| Pickle          | Good           |
| Secret Dressing | NULL           |
| Tomato          | Good           |
| Water           | NULL           |
| Soda            | NULL           |
| Watermelon      | Good           |
| Orange          | Good           |



# Reshaping Results

```
SELECT name,  
       FLOOR(  
         CASE  
           WHEN inventory < 20 THEN 20 - inventory  
           WHEN foodgroup = 'Milk' THEN inventory * 0.05  
           WHEN foodgroup IN ('Meat', 'Bread') THEN inventory * 0.10  
           WHEN foodgroup = 'Vegetable' AND unitprice <= 0.03 THEN inventory * 0.10  
           WHEN foodgroup = 'Vegetable' THEN inventory * 0.03  
           WHEN foodgroup = 'Fruit' THEN inventory * 0.04  
           WHEN foodgroup IS NULL THEN inventory * 0.07  
           ELSE 0  
         END) AS size, vendorid  
FROM ingredients  
WHERE inventory < 1000 AND vendorid IS NOT NULL  
ORDER BY vendorid, size;
```

| name    | size | vendorid |
|---------|------|----------|
| Cheese  | 7    | DNDRY    |
| Chicken | 12   | DNDRY    |
| Crouton | 40   | EDDRS    |
| Orange  | 10   | FRTFR    |
| Grape   | 12   | FRTFR    |
| Tomato  | 5    | VGRUS    |
| Lettuce | 20   | VGRUS    |
| Pickle  | 24   | VGRUS    |



# Reshaping Results

- NULLIF

```
SELECT ingredientid, name, unit, unitprice,  
NULLIF(foodgroup, 'Meat') AS foodgroup,  
inventory, vendorid FROM ingredients;
```

| ingredientid | name            | unit  | unitprice | foodgroup | inventory | vendorid |
|--------------|-----------------|-------|-----------|-----------|-----------|----------|
| CHESE        | Cheese          | scoop | 0.03      | Milk      | 150       | DNDRY    |
| CHIKN        | Chicken         | strip | 0.45      | NULL      | 120       | DNDRY    |
| CRUTN        | Crouton         | piece | 0.01      | Bread     | 400       | EDDRS    |
| GRAPE        | Grape           | piece | 0.01      | Fruit     | 300       | FRTFR    |
| LETUS        | Lettuce         | bow1  | 0.01      | Vegetable | 200       | VGRUS    |
| PICKL        | Pickle          | slice | 0.04      | Vegetable | 800       | VGRUS    |
| SCTOR        | Secret Dressing | ounce | 0.03      | NULL      | 120       | NULL     |
| TOMTO        | Tomato          | slice | 0.03      | Fruit     | 15        | VGRUS    |
| WATER        | Water           | glass | 0.06      | NULL      | NULL      | SPWTR    |
| SODA         | Soda            | glass | 0.69      | NULL      | 5000      | SPWTR    |
| WTRML        | Watermelon      | piece | 0.02      | Fruit     | NULL      | FRTFR    |
| ORNG         | Orange          | slice | 0.05      | Fruit     | 10        | FRTFR    |



# Reshaping Results

- COALESCE

```
SELECT name, price, COALESCE(price, 0.00) AS "no nulls"  
FROM items;
```

| name             | price | no nulls |
|------------------|-------|----------|
| Chicken Salad    | 2.85  | 2.85     |
| Fruit Salad      | 3.45  | 3.45     |
| Garden Salad     | 0.99  | 0.99     |
| Millennium Salad | NULL  | 0.00     |
| Soda             | 0.99  | 0.99     |
| Water            | 0.00  | 0.00     |
| Fruit Plate      | 3.99  | 3.99     |



"THANK YOU  
FOR YOUR  
ATTENTION"

